

shroomio Software Specification

Formal Product Requirements, Architecture, and Engineering Baseline

shroomio Authors

June 11, 2026

Contents

1	Document Purpose	3
2	Product Overview	3
2.1	Product Identity	3
2.2	Core Loop	3
2.3	Design Pillars	4
3	Architecture	4
3.1	Module Map	4
3.2	Dependency Graph	5
3.3	Build System	5
4	Data Structures	6
4.1	World State	6
4.2	Player State	6
4.3	Spore State	6
4.4	Zone Enumeration	6
5	Protocol Specification	7
5.1	Transport	7
5.2	Common Header	7
5.3	Packet Types	7
5.3.1	Hello (client to server, type = 1)	7
5.3.2	Welcome (server to client, type = 2)	8
5.3.3	Input (client to server, type = 3)	8

5.3.4	Snapshot (server to client, type = 4)	8
5.3.5	Ping / Pong (type = 5, 6)	9
5.4	Snapshot Capacity Constraint	9
6	Simulation Rules	9
6.1	World	9
6.2	Movement	9
6.3	Spore Collection	10
6.4	Consumption	10
6.5	Spawning and Respawning	10
6.6	Bot AI	11
6.7	Tuning Parameters	11
7	Client Specification	11
7.1	Rendering	11
7.2	HUD	12
7.3	Input	12
7.4	Dual-Mode Operation	12
7.5	Network Client Lifecycle	12
8	Server Specification	13
8.1	Startup	13
8.2	Tick Loop	13
8.3	Session Management	13
8.4	Snapshot Broadcast	13
8.5	Deployment	14
9	Engineering Review and Risk Assessment	14
9.1	Code Quality	14
9.2	Identified Gaps and Risks	14
10	Engineering Roadmap	15
10.1	Milestone 1: Online Spore Economy (G1)	15
10.2	Milestone 2: Snapshot Scaling (G2)	15
10.3	Milestone 3: Client-Side Smoothing (G3)	16
10.4	Milestone 4: Player Identity and Session UX (G4, G5)	16

10.5 Milestone 5: Polish and Content (G6, G7, G8, G9)	16
11 Acceptance Criteria	16
12 Non-Goals for the Current Phase	17
13 Document Maintenance	17

1 Document Purpose

This document is the formal software specification for **shroomio**, a real-time multiplayer fungal arena game written in C. It replaces the earlier informal *Product Requirements and Technical Baseline* document and constitutes the single authoritative reference for all product requirements, technical architecture, protocol design, balancing parameters, and engineering roadmap.

This specification draws from three sources:

- the actual implementation as it exists in the repository today,
- the intended product vision described in the earlier baseline document,
- a formal engineering review of the current codebase against that vision.

Where a requirement is already satisfied by the codebase it is annotated with the relevant source file. Where a requirement is not yet implemented it is marked as a scheduled milestone.

2 Product Overview

2.1 Product Identity

shroomio is a real-time multiplayer arena game in which players control fungal organisms, collect spores to grow, consume smaller organisms, avoid larger threats, and compete on a mass-based leaderboard. The game is designed to be playable both locally (offline against bots) and online (against human opponents via a headless dedicated server).

2.2 Core Loop

The intended player loop is:

1. enter the arena at low-to-mid mass,
2. collect safe spores in the outer ring,
3. rotate toward mid and center zones to improve growth rate,
4. watch for favorable fights against smaller targets,

5. avoid larger predators as increased mass lowers mobility,
6. climb the leaderboard,
7. respawn quickly and re-engage after defeat.

2.3 Design Pillars

- **Readable competition:** player state and arena risk must be legible at a glance.
- **Fast re-entry:** defeat resets the player quickly without removing them from the session.
- **Authoritative multiplayer:** server state is the source of truth for movement and collisions.
- **Low content overhead:** the game remains fun with simple shapes, simple rules, and lightweight assets.
- **Incremental extensibility:** simulation, client rendering, and server networking stay separated so the prototype can evolve safely.

3 Architecture

3.1 Module Map

The repository is organized into four top-level modules, each with a defined responsibility and dependency boundary.

Module	Source Path	Responsibility
Client	<code>src/client/main.c</code>	Application entry, window creation, render loop dispatch
Client / Game	<code>src/client/game.c</code> , <code>game.h</code>	Game state initialization, update tick (local or network), all 2D rendering (arena, players, spores, UI), camera management, input collection
Client / Net	<code>src/client/net.c</code> , <code>net.h</code>	ENet host lifecycle, hello/welcome handshake, per-tick input sending, snapshot reception and storage
Server	<code>src/server/main.c</code>	ENet host lifecycle, session management, hello/input/snapshot dispatch, tick scheduler, signal handling
Shared / Config	<code>src/shared/config.h</code>	Compile-time tuning constants (world size, speeds, spore count, consume ratios, tick rate)
Shared / World	<code>src/shared/world.h</code>	Data-type definitions: <code>ShroomWorldState</code> , <code>ShroomPlayerState</code> , <code>ShroomSporeState</code> , <code>ShroomZone</code>

Shared / Sim	src/shared/sim.c, sim.h	Deterministic simulation: movement, spore collection, consume resolution, bot AI, spawn/respawn logic
Shared / Protocol	src/shared/protocol.h	Binary packet format definitions for all client-server messages
Shared / Vector	src/shared/vec2.h	Inline 2D vector math

3.2 Dependency Graph

client/main.c

```
-> client/game.h -> client/net.h -> shared/protocol.h -> shared/vec2.h
    -> shared/world.h -> shared/config.h -> shared/vec2.h
    -> raylib
```

server/main.c

```
-> shared/sim.h -> shared/world.h -> shared/config.h -> shared/vec2.h
-> shared/protocol.h
-> enet
```

shared/sim.c

```
-> shared/sim.h
-> shared/world.h
```

Rule: client code may depend on shared code but never on server code. Server code may depend on shared code but never on client code. Shared code has no dependency on either client or server, and no dependency on `raylib` or `enet`.

3.3 Build System

The project uses a single GNU Makefile at the repository root.

- `make linux` — builds the Linux raylib client binary (`dist/shroomio`).
- `make server` — builds the Linux headless server binary (`dist/shroomio-server`).
- `make windows` — cross-compile the Windows client binary (`dist/shroomio.exe`) using mingw-w64.
- `make run` — builds and launches the Linux client.
- `make run-server` — builds and launches the headless server.
- `make docker-server` — builds the server Docker image.
- `make docker-run-server` — builds and runs the server Docker container.

The Makefile downloads raylib source (version 5.5) on demand, compiles `libraylib.a` statically for each target, and bundles ENet (vendored under `vendor/enet/`) for the server target.

Supported platforms:

- Linux client (gcc/clang, raylib static, system GL/X11 libs dynamic)
- Linux headless server (gcc/clang, no graphics libraries)
- Windows client cross-build from Linux (mingw-w64, fully static exe)

4 Data Structures

4.1 World State

```
ShroomWorldState {
    uint64_t      tick;
    float         width, height;           // 6000 x 6000
    ShroomEntityId next_entity_id;
    size_t        player_count;           // <= SHROOM_MAX_PLAYERS (128)
    size_t        spore_count;            // <= SHROOM_MAX_SPORES (4096)
    ShroomPlayerState players[128];
    ShroomSporeState spores[4096];
}
```

4.2 Player State

```
ShroomPlayerState {
    ShroomPlayerId player_id;           // unique player identifier
    ShroomEntityId entity_id;           // unique entity identifier
    ShroomVec2      position;            // world-space position
    ShroomVec2      input_direction;     // normalized or zero
    float           mass;                 // current mass
    float           radius;               // derived from mass
    bool            alive;
    bool            is_bot;
}
```

4.3 Spore State

```
ShroomSporeState {
    ShroomEntityId entity_id;
    ShroomVec2      position;
    uint16_t        value;                // mass granted on collection (8)
    bool            active;
}
```

4.4 Zone Enumeration

```
ShroomZone {
```

```

    SHROOM_ZONE_OUTER = 0,
    SHROOM_ZONE_MID    = 1,
    SHROOM_ZONE_CENTER = 2,
}

```

Zones are concentric circles centered on the world midpoint. Center radius is 900, mid radius is 2000, outer is everything beyond mid up to the world border.

5 Protocol Specification

5.1 Transport

All multiplayer communication uses ENet over UDP on port 7777. ENet provides ordered reliable delivery on channel 0 (control), and unreliable sequenced delivery on channels 1 (input) and 2 (snapshot).

Channel	ID	QoS	Purpose
Control	0	Reliable, ordered	Hello, Welcome, Ping, Pong
Input	1	Unreliable, sequenced	Player directional input
Snapshot	2	Unreliable, sequenced	Server-to-client world state updates

5.2 Common Header

Every packet begins with a 4-byte header:

```

ShroomPacketHeader {
    uint8_t  type;          // ShroomPacketType enum
    uint8_t  reserved;
    uint16_t size;          // total packet size in bytes
}

```

5.3 Packet Types

5.3.1 Hello (client to server, type = 1)

```

ShroomHelloPacket {
    ShroomPacketHeader header;
    uint32_t protocol_version; // SHROOM_PROTOCOL_VERSION (1)
    char     name[32];         // player-chosen display name
}

```

Sent immediately after ENet connection. If `protocol_version` does not match, the server disconnects the peer. If the server is full, the peer is disconnected.

5.3.2 Welcome (server to client, type = 2)

```
ShroomWelcomePacket {
    ShroomPacketHeader header;
    uint32_t protocol_version;
    uint32_t player_id;
    uint32_t entity_id;
    uint16_t server_tick_rate;    // 30
    uint16_t snapshot_rate;      // 15
    float    world_width;        // 6000.0
    float    world_height;       // 6000.0
}
```

Sent in response to a valid Hello. After receiving Welcome, the client switches from local simulation to applying server snapshots.

5.3.3 Input (client to server, type = 3)

```
ShroomInputPacket {
    ShroomPacketHeader header;
    uint32_t sequence;          // monotonic, client-side
    float    direction_x;       // normalized input x
    float    direction_y;       // normalized input y
}
```

Sent per tick at the server tick rate (30 Hz). The server clamps and normalizes the direction vector before applying it.

5.3.4 Snapshot (server to client, type = 4)

```
ShroomSnapshotPacket {
    ShroomPacketHeader    header;
    uint64_t              tick;
    uint32_t              last_processed_input_sequence;
    uint32_t              player_id;          // this client's player
    uint32_t              entity_id;
    uint16_t              player_count;
    uint16_t              reserved;
    ShroomSnapshotPlayerState players[32];
}
```

```
ShroomSnapshotPlayerState {
    uint32_t player_id;
    uint32_t entity_id;
    float    position_x;
    float    position_y;
    float    mass;
}
```

```

    float    radius;
    uint8_t   alive;
    uint8_t   is_bot;
    uint16_t  reserved;
}

```

Broadcast at the snapshot rate (15 Hz). Contains up to 32 player entries. Clients overwrite their local player array from this payload.

Known limitation: spores are not included in snapshot packets. Online players do not see or interact with spores through the network layer. Spore replication is milestone 1 in the engineering roadmap (§10).

5.3.5 Ping / Pong (type = 5, 6)

```

ShroomPingPacket { ShroomPacketHeader; uint32_t nonce; }
ShroomPongPacket { ShroomPacketHeader; uint32_t nonce; }

```

Used for latency measurement. The server echoes Pong with the same nonce. Not yet wired to a visible RTT display on the client.

5.4 Snapshot Capacity Constraint

The simulation supports up to 128 players (`SHROOM_MAX_PLAYERS`), but the snapshot packet is hard-coded to 32 player entries (`SHROOM_MAX_SNAPSHOT_PLAYERS`). This means that in a full online match, only the first 32 alive players will be visible to any given client. This is a known capacity gap scheduled for remediation in milestone 2 (§10).

6 Simulation Rules

6.1 World

- World dimensions: 6000×6000 units (square).
- Entity positions are clamped to the rectangle $[radius, width - radius] \times [radius, height - radius]$.
- Three concentric zones centered at (3000, 3000): center ($r \leq 900$), mid ($900 < r \leq 2000$), outer ($r > 2000$).

6.2 Movement

- Each tick, each alive player is moved by $velocity = input_direction \times speed \times \Delta t$.
- Input direction is a normalized 2D vector (zero allowed for no movement).
- Speed is derived from mass:

```
speed = SHROOM_MAX_PLAYER_SPEED / (1.0 + (mass * 0.35 / 100.0))
clamp(speed, SHROOM_MIN_PLAYER_SPEED, SHROOM_MAX_PLAYER_SPEED)
```

- Minimum speed: 120. Maximum speed: 260. (Both units/sec.)
- Bots compute their own input direction each tick (see §6.6).

6.3 Spore Collection

- The world maintains a target of 900 active spores.
- Each spore has a value of 8 mass.
- Collection radius is `player_radius + 6.0` units.
- When a player overlaps a spore, the player gains the spore's mass value and the spore is immediately relocated to a new random position (biased toward center and mid zones).

Spore spawn bias (per `ShroomSpawnOrResetSpore`):

- 45% chance: placed within the center zone.
- 35% chance: placed within the mid zone (beyond center but within mid radius).
- 20% chance: placed anywhere in the world (outer).

6.4 Consumption

- A player may consume another player if:
 1. both are alive,
 2. the attacker's mass is at least $1.15\times$ the target's mass,
 3. the attacker's center overlaps the target's radius scaled by 0.88.
- On consumption: the attacker gains $0.70\times$ the victim's mass; the victim respawns immediately at default mass (100) in a safe outer-zone location.
- Multiple simultaneous consumes are resolved in a single pass: a victim is consumed at most once per tick.

6.5 Spawning and Respawning

- New players and respawning players are placed in the outer zone, at least 340 units away from any alive player.
- Up to 32 random candidate positions are tried before falling back to a random position without safety checks.
- Default spawn mass is 100.
- Radius is recalculated from mass: $\text{radius} = 10.0 + \text{mass} \times 0.14$.

6.6 Bot AI

Bots follow a priority-ordered behavior tree evaluated each tick:

1. **Flee**: if a larger player ($\text{mass} > \text{bot_mass} \times 1.15$) is within 900 units, move directly away.
2. **Chase**: if a smaller player ($\text{bot_mass} > \text{target_mass} \times 1.15$) is within 700 units, move directly toward.
3. **Forage**: move toward the nearest active spore.
4. **Drift**: move toward the world center.

6.7 Tuning Parameters

Parameter	Value	Defined In
World width / height	6000×6000	config.h:7-8
Max players (sim)	128	config.h:4
Max spores	4096	config.h:5
Max snapshot players	32	protocol.h:15
Center zone radius	900	config.h:10
Mid zone radius	2000	config.h:11
Default player mass	100	config.h:13
Bot count	18	config.h:14
Min player speed	120	config.h:15
Max player speed	260	config.h:16
Speed mass scale	0.35	config.h:17
Spore target count	900	config.h:19
Spore value	8	config.h:20
Consume mass advantage	1.15	config.h:22
Consume mass gain factor	0.70	config.h:23
Spawn safe distance	340	config.h:24
Server tick rate	30 Hz	config.h:26
Snapshot rate	15 Hz	protocol.h:14
Server port	7777 / UDP	protocol.h:7
Protocol version	1	protocol.h:6
Max player name length	32	protocol.h:8

7 Client Specification

7.1 Rendering

- Uses **raylib** 5.5 with a 2D camera (**Camera2D**) centered on the local player.
- Target window: 1280×720 at 60 FPS.
- Arena zones rendered as colored rectangles/circles with alpha blending.

- Player entities rendered as filled circles, color-coded: human players in green ($\{126, 217, 87\}$), bots in preset palette colors rotated by player ID.
- Local player highlighted with an additional white outline ring.
- Spores rendered as small gold circles (radius 4).
- Grid drawn every 80 units for spatial orientation.
- World border drawn as a rectangle outline.

7.2 HUD

The client renders two fixed HUD panels (top-left and top-right) regardless of camera position:

- **Top-left:**
 - Title.
 - Movement hint (“Move toward the mouse...”).
 - Mass, radius, current speed.
 - Zone label, player count, spore count, tick.
 - Server connection status and last snapshot tick (green when connected, orange otherwise).
- **Top-right:** leaderboard, top 6 entries, highlighting the local player with “You” label and white text.

7.3 Input

Primary movement: mouse-to-world vector (screen to world using `GetScreenToWorld2D`). Secondary: WASD and arrow keys add $+1.0$ or -1.0 to the accumulated direction vector. The resulting vector is normalized before being sent to the simulation or the server.

7.4 Dual-Mode Operation

- **Offline mode:** the client runs the shared simulation (`ShroomWorldStep`) locally. No network connection required. Full spore economy and bot AI run client-side.
- **Online mode:** after a successful Welcome handshake, the client stops running `ShroomWorldStep` locally and instead applies snapshots from the server (`ApplyNetworkSnapshot`). Spore state is zeroed in this mode.

7.5 Network Client Lifecycle

1. `ENet init`
2. `Create ENet client host`
3. `Resolve server address`
4. `Connect to server`

5. Wait for ENet connect event -> send Hello
6. Wait for Welcome -> status = CONNECTED
7. Send Input packets each tick at 30 Hz
8. Receive Snapshot packets at ~15 Hz
9. On disconnect -> reset to local offline mode

8 Server Specification

8.1 Startup

1. Configure stdout/stderr as unbuffered
2. Install SIGINT / SIGTERM handlers
3. `enet_initialize()`
4. `ShroomWorldInit()` -> spawn 18 bots
5. `enet_host_create(&address, 128 clients, 3 channels)`
6. Enter tick loop

8.2 Tick Loop

- Fixed-timestep loop at 30 Hz using `clock_gettime(CLOCK_MONOTONIC)` and `nanosleep`.
- Each tick:
 1. Service all ENet events (connect, receive, disconnect).
 2. Call `ShroomWorldStep(world, 1/30)`.
 3. If tick is divisible by the snapshot interval (every 2nd tick), broadcast snapshots to all connected peers.
 4. Flush ENet.
 5. Sleep until next tick boundary.

8.3 Session Management

- Each connected peer is assigned a `ServerSession` slot (max 128).
- On connect: the session is stored via `peer->data`.
- On Hello: session is activated, player spawned, Welcome sent.
- On Input: the input sequence is recorded and direction applied to the player.
- On disconnect: the player is zeroed (mass = 0, radius = 0, alive = false) and the session deactivated.

8.4 Snapshot Broadcast

Snapshots are sent to every connected peer. The snapshot payload includes the first 32 alive players with positive mass. If more than 32 alive players exist, only the first 32 are included.

8.5 Deployment

The server is packaged as a Docker container:

```
FROM debian:bookworm-slim AS builder
# install build-essential, make
# COPY Makefile, src/, vendor/enet
# RUN make server

FROM debian:bookworm-slim
# COPY shroomio-server binary
# EXPOSE 7777/udp
# CMD ["/usr/local/bin/shroomio-server"]
```

Run with:

```
make docker-server      # build image
make docker-run-server  # build and run
docker compose up --build server # via compose
```

9 Engineering Review and Risk Assessment

9.1 Code Quality

The implementation is compact (12 source files, approximately 1 400 lines of C), well-structured, and consistent. The separation of shared simulation from client and server is maintained without violation. No known memory leaks or undefined behavior; the code uses stack-allocated structs and fixed-size arrays throughout with no dynamic allocation beyond ENet internals.

9.2 Identified Gaps and Risks

ID	Gap	Severity	Mitigation
G1	Spores not replicated in snapshots; online players cannot collect spores	High	M1: add spore replication to snapshot protocol
G2	Snapshot max 32 players vs simulation max 128; invisible players in full matches	High	M2: expand snapshot capacity or add interest management
G3	No client-side prediction, interpolation, or reconciliation; movement will feel jittery under latency	Medium	M3: add interpolation buffer and dead reckoning
G4	Player name sent in Hello but not stored or displayed anywhere	Medium	M4: store names server-side, include in snapshots, render in HUD

G5	No menu, server browser, or connection UX beyond status text	Low	M4: add basic connection screen
G6	No audio, sprites, or themed content (scaffold folders exist but are empty)	Low	M5: art pass after core loop is network-complete
G7	Deterministic PRNG seeded with constant (<code>srand(7)</code>); identical bot/spore patterns every session	Low	Seed from time or allow config override
G8	No RTT display on client despite ping/pong support	Low	Wire ping RTT to HUD status line
G9	No CI, automated testing, or linting in repository	Medium	Add CI pipeline with build verification
G10	README duplicates content (last lines)	Low	Clean README

10 Engineering Roadmap

10.1 Milestone 1: Online Spore Economy (G1)

Add spore state to server snapshots so online matches include the full growth loop.

- Extend `ShroomSnapshotPacket` or add a new `ShroomWorldStatePacket` type that includes spore data.
- Update `ApplyNetworkSnapshot` to populate `world.spores` from network data.
- Ensure spore collection is resolved authoritatively on the server and the client applies server spore state.
- Files affected: `protocol.h`, `server/main.c`, `client/net.c`, `client/game.c`.

10.2 Milestone 2: Snapshot Scaling (G2)

Increase visible player capacity beyond 32.

- Either: raise `SHROOM_MAX_SNAPSHOT_PLAYERS` to 128 (packet size $\approx$ 5 KiB, acceptable over LAN/DC).
- Or: implement distance-based interest management to send only the nearest N players.
- Files affected: `protocol.h`, `server/main.c`, `client/net.c`.

10.3 Milestone 3: Client-Side Smoothing (G3)

Add interpolation and basic prediction for smooth perceived motion.

- Buffer the last two snapshots and interpolate entity positions between them.
- Apply dead reckoning for the local player between input sends.
- Files affected: `client/game.c`, `client/net.c`.

10.4 Milestone 4: Player Identity and Session UX (G4, G5)

- Store player name server-side in `ServerSession`.
- Include name in `ShroomSnapshotPlayerState` (add 32-byte name field).
- Render player names above entities in the client.
- Add a basic text-entry prompt for player name on client launch.
- Add a simple connection screen (server address entry, connect/disconnect button).
- Files affected: `protocol.h`, `server/main.c`, `client/net.c`, `client/game.c`, `client/main.c`.

10.5 Milestone 5: Polish and Content (G6, G7, G8, G9)

- Seed PRNG from `time(NULL)` or add a config option.
- Display RTT in client HUD status line.
- Add basic CI (GitHub Actions) that runs `make linux` and `make server`.
- Clean README.
- Art direction pass: simple sprite rendering, audio stubs.

11 Acceptance Criteria

The current product slice is considered successful when:

- a player can launch the client and move immediately,
- the player can grow via spore collection,
- larger entities can consume smaller ones under deterministic rules,
- the player respawns quickly after defeat,
- a headless server can host the simulation on UDP port 7777,
- a client can connect, complete handshake, submit input, and render server snapshots,

- this specification document can be regenerated from the repository using `latexmk`.

Each milestone above extends this baseline with additional, verifiable criteria documented in the corresponding GitHub issue.

12 Non-Goals for the Current Phase

- Deep account systems or progression.
- Monetization design.
- Matchmaking services beyond direct local or hosted connection.
- High-fidelity art requirements before the core loop is proven.
- Large-scale persistent world simulation.

13 Document Maintenance

This specification is the authoritative reference for the shroomio project. It must be updated whenever:

- a tuning constant changes in `config.h` or `protocol.h`,
- a new packet type or channel is introduced,
- the data model is extended,
- a roadmap milestone is completed,
- a new architectural module is added.

The source file lives at `design/shroomio-specification.tex` and is compiled with `latexmk -pdf` inside the devcontainer.